Guía de pruebas antes de la demo final

PIDS 2026 · Proyecto 2 · Grupo 2 · Bloque 3 (integración en el RoboMaster) · 04/11/2026

Esta guía ordena las pruebas desde la de menos riesgo (sin robot) hasta la demo completa. Cada paso dice qué ejecutar, qué debe pasar y qué anotar. Todo se lanza desde la carpeta `ENTREGA 2/WEEK3` del repositorio; los detalles del código están en su `README.md`.

Normas de seguridad (enunciado del Bloque 3)

- Está **prohibido aumentar la velocidad** del robot. La demo nunca pasa de 0,5 m/s ni de 30 º/s.
- Cuando el robot se mueva, un compañero lo sigue de cerca. Ante cualquier riesgo, **Ctrl+C**: la demo para el chasis, centra la torreta y apaga los LED.
- Deje espacio en todas direcciones y no confunda los ejes x e y (figura de la sección 5).
- Coja el robot **siempre por la base**, nunca por la cabeza. No apunte a nadie con el blaster.

Orden de las pruebas

Paso	Qué	Riesgo	Sección
1	Preparar el ordenador, el entorno de Python 3.8 y el modelo	Ninguno	1
2	Conectar el robot (wifi directo o networking)	Ninguno	2
3	Scripts de prueba de cada módulo	Bajo	3
4	Pruebas automáticas y demo simulada, sin robot	Ninguno	4
5	Medir el circuito y rellenar circuito.py	Ninguno	5
6	Calibrar cada parada: sin mover el chasis ni disparar	Bajo	6
7	Recorrido completo sin disparar	Medio	7
8	Demo final	Medio	8
9	Guardar las evidencias para la memoria	Ninguno	9

1. Preparación

1. Abra en VSCode la carpeta `ENTREGA 2/WEEK3` (Archivo → Abrir carpeta). Así los scripts se ejecutan desde ella y encuentran sus ficheros.
2. Intérprete (Ctrl+Shift+P → «Python: Select Interpreter»): en el portátil, `.venv/Scripts/python.exe` de esa carpeta; en los ordenadores del laboratorio, `/home/PIDS/robomaster/.pixi/venv/bin/python`. El SDK solo funciona con Python 3.8; con otro intérprete sale `No module named robomaster`.
3. Ejecute **sin depurador**: Ctrl+F5 o la flecha junto al botón Play → «Run Python File». Al relanzar con el depurador, VSCode da a veces `ConnectionRefusedError` sin ejecutar el script.
4. Modelo: `models/faster_rcnn_v2_aug_robusto_v1_lr001/model_demo.pt` (173 MB, solo los pesos del modelo final). No se sube a GitHub; el README explica cómo exportarlo de JupyterHub.
5. Batería del robot cargada y blaster revisado (si lleva bolitas de gel, cuidado con los disparos de agua).

En los comandos de esta guía, **PY** es el intérprete: en el portátil con Windows, `.venv\Scripts\python.exe`; en el laboratorio, `python`.

2. Conectar el robot

Modo	Cuándo	Cómo
Wifi directo	Portátil con wifi	Pestaña de conexión del robot en el icono de wifi directo. El portátil se conecta a la red **RMEP-…** del robot (contraseña en su pegatina, suele ser 12341234). No se usa connect.py. Mientras tanto el portátil no tiene internet.
Networking	Ordenadores del laboratorio (sin wifi)	Rellenar en connect.py la SSID, la contraseña del wifi del laboratorio y el número de serie, **solo en local**. Pestaña en el icono del router, pulsar el botón rojo, lanzar **PY connect.py** y enseñar el QR a la cámara cogiendo el robot por la base. Crea serial_number.txt.

Todos los scripts eligen el modo solos: con serial_number.txt en la carpeta usan networking; sin él, wifi directo. Al conectar escriben `Conexion: wifi` o `Conexion: networking`. Para volver a wifi directo después de usar connect.py, borre serial_number.txt.

> **Nunca se suben al repositorio**
>
> - La SSID, la contraseña del wifi ni el número de serie de connect.py; qrcode.png y serial_number.txt están ignorados. En el portátil de Javier, connect.py está marcado para que git no vea sus cambios.
> - El modelo (.pt), la carpeta evidencias/ ni el entorno .venv/.

3. Scripts de prueba de cada módulo

Láncelos en este orden. Cada uno se conecta, hace una cosa y se desconecta. Anote lo de la última columna: se usa para ajustar circuito.py.

#	Comando (PY …)	Qué hace	Qué comprobar	Qué anotar
1	others\battery.py	Lee la batería y ajusta el brillo de los LED	Sale «Battery: N%»	Cargar si < 20 %
2	others\led.py	Sube el brillo y acaba en morado	Se encienden todos los LED	—
3	audio\predefined_sounds.py	Sonidos 0 a 19, uno cada 3 s (1 min)	Se oyen	Números que gusten (nota A)
4	camera\stream.py	10 s de vídeo en directo	Imagen nítida y sin cortes	—
5	camera\record.py	Graba 200 fotogramas en recording.mp4	Se crea el vídeo	Grabar el circuito sirve para la memoria
6	gimbal\gimbal_speed.py	Gira la torreta a velocidad constante	Movimiento suave	—
7	gimbal\gimbal_degrees.py	Torreta a yaw +90 y −90, luego pasos de 30º	**¿yaw +90 mira a la derecha?**	Si mira a la izquierda: nota B
8	chassis\chassis_speed.py	Adelante, atrás, a cada lado y giros, 3 s cada uno	Deje 2 m libres	—
9	chassis\chassis_distance.py	1 m adelante y atrás, 1 m a cada lado, giro de 90º y vuelta	**¿y < 0 va a la izquierda? ¿z > 0 gira a la izquierda?** Mida con cinta si recorre 1 m	Error de distancia y de giro (nota B)
10	others\blaster.py	1 y 3 disparos de agua; 1 y 3 de infrarrojo	Apunte a una pared	—
11	audio\play.py (opcional)	Reproduce engine_sound.wav	Lanzar desde la carpeta audio	—

Notas

- **A. Sonidos.** La demo usa los sonidos 0x101–0x106 del SDK (cuenta atrás, escaneo, detectado, disparo, hecho). Si en la demo no suenan, ponga en SONIDOS de circuito.py los números que oyó en el script 3.
- **B. Sentidos.** circuito.py supone yaw > 0 a la derecha, y > 0 a la derecha y z > 0 giro a la izquierda. Si la torreta va al revés: SIGNO_YAW = -1. Si el chasis va al revés, cambie los signos de las paradas. Si recorre más o menos de 1 m, téngalo en cuenta al medir las paradas.
- Las velocidades de los ejemplos del chasis y la torreta están ya bajadas a las del SDK (0,5 m/s y 30 º/s).

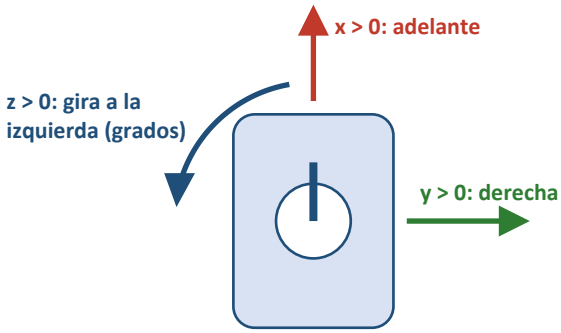
4. Pruebas sin robot

Comprueban que el código y el modelo funcionan en ese ordenador antes de tocar el robot.

```
PY -m unittest discover -s tests
PY demo.py --simulacion
```

- La primera debe acabar en `Ran 15 tests ... OK`. Prueban con un detector falso la puntería, los 12 disparos, que no se repiten robots, las velocidades, Ctrl+C y la sintaxis de Python 3.8.
- La segunda carga el modelo final (`Modelo faster_rcnn_v2 en cpu`) y hace el recorrido con imágenes del test de la asignatura, con un robot simulado. Anote el tiempo de cada imagen: en una CPU de 32 hilos es de unos 1,1 s. Dentro del repositorio no hace falta indicar carpeta; fuera, `--simulacion CARPETA` con imágenes.

5. Medir el circuito y rellenar circuito.py



robot (vista desde arriba)

Chasis
move(x, y, z): x adelante, y derecha, z giro a la izquierda. Movimientos relativos a donde está.

Gimbal (torreta)
yaw > 0 mira a la derecha; pitch > 0 mira arriba.

Compruébelo con los scripts 7 y 9 de la tabla: si algo va al revés, ver la sección 3.

Cada parada de la lista PARADAS es un diccionario. Los movimientos son relativos al final de la parada anterior y se ejecutan en orden. Ejemplo:

```
PARADAS = [
    # avanza 1 m y mira de frente y a ±45º; se espera 1 robot
    dict(nombre='P1', movimientos=[(1.0, 0, 0)],
         barrido=BARRIDO_FRONTAL, pitch=0, objetivos=1),
    # gira 90º a la izquierda, avanza 1 m y barre 360º
    dict(nombre='P2', movimientos=[(0, 0, 90), (1.0, 0, 0)],
         barrido=BARRIDO_360, pitch=0, objetivos=1),
    # no se sabe cuántos hay: barrido completo
    dict(nombre='P3', movimientos=[(0, 0, -90), (0.8, 0, 0)],
         barrido=BARRIDO_360, pitch=0, objetivos=None),
]
```

Campo	Qué es
nombre	Aparece en el nombre de las imágenes de evidencias (P1, P2…)

Campo	Qué es
movimientos	Lista de (x, y, z): metros adelante, metros a la derecha, grados de giro a la izquierda
barrido	Ángulos de la torreta donde se hace una foto. BARRIDO_360 = 0, 60, 120, 180, −120, −60; BARRIDO_FRONTAL = 0, −45, 45; o una lista propia
pitch	Inclinación de la torreta (grados). 0 suele valer; algo negativo si los robots están cerca
objetivos	Robots que se esperan desde ahí. Al atacarlos deja de girar; None = barrido completo

- Elija paradas con buena visibilidad de las posibles posiciones de los rivales y a más de 30 cm de los obstáculos.
- Prefiera movimientos simples: girar parado y luego avanzar en línea recta.
- Otros ajustes: UMBRAL (0,95, fijado en validación), DISPARAR_A_TODOS, BARRIDO_COMPLETO, VELOCIDAD_XY (0,3 m/s; se puede bajar, nunca subir), VOLVER_AL_INICIO y BAILE_FINAL.

6. Calibrar cada parada

Coloque el robot a mano en la parada, orientado como llegaría, y lance:

```
PY demo.py --sin-mover --sin-disparar --paradas P1
```

- La torreta barre, detecta y apunta, pero ni el chasis se mueve ni el blaster dispara.
- Abra la carpeta evidencias/<fecha_hora>/. Cajas **verdes**: aceptadas (≥ umbral). **Naranjas**: vistas pero por debajo del umbral. La **cruz roja** es el objetivo.
- En las imágenes «confirmado», la cruz debe quedar en la línea vertical del centro. Si cae siempre hacia el mismo lado, la torreta gira al revés (SIGNO_YAW = -1).
- Si los rivales salen en naranja con 0,8–0,94, el umbral es demasiado estricto con la luz del laboratorio: pruebe `--umbral 0.9` y apunte el motivo para la memoria. No baje el umbral si aparecen falsos positivos en verde.
- Repita con cada parada (--paradas P2, P3…).

7. Recorrido completo sin disparar

```
PY demo.py --sin-disparar
```

- Con un compañero junto al robot y Ctrl+C a mano. La consola escribe cada movimiento antes de hacerlo (`chasis: x=+1.00 m ...`).
- Compruebe que llega a cada parada sin rozar obstáculos y corrija las distancias en circuito.py.
- Para probar solo el principio del recorrido: --paradas P2 hace los movimientos hasta P2 y se para ahí.

8. Demo final

```
PY demo.py
```

Qué hace en cada parada: se desplaza, barre con la torreta, apunta a cada robot nuevo, vuelve a detectar para confirmar y corregir la puntería, dispara 12 veces y cabecea. Si al confirmar el robot no está centrado o es uno ya atacado, no dispara. Al final, baile. Significado de los LED:

Color	Estado
Azul	En marcha (desplazamiento)
Ámbar	Buscando (barrido)
Naranja	Apuntando
Rojo parpadeando	Disparando
Verde	Robot inutilizado
Morado	Parada terminada sin encontrar robots
Verde agua	Demo terminada

9. Evidencias para la memoria

Cada ejecución crea evidencias/<fecha_hora>/ con:

Fichero	Contenido
NNN_parada_fase_yawXX.jpg	Cada imagen a la que se aplicó el detector, con las cajas y la confianza. Fases: barrido, confirmado, sin_confirmar
originales/	Las mismas imágenes sin dibujar (para anotarlas o reentrenar)
registro.csv	Una fila por imagen: parada, yaw, tiempo de inferencia, confianzas, cajas, ángulo del objetivo y disparos
resumen.json	Configuración, robots atacados por parada, disparos totales, tiempo medio y duración

- Guarde la carpeta de cada intento (también los fallidos) y anote qué pasó en cada uno: el enunciado valora más una demo imperfecta bien analizada que una perfecta sin analizar.
- Para la memoria: robots vistos y no vistos por parada, falsos positivos, disparos, tiempo por imagen, efecto de la luz y de la distancia, y cambios de umbral con su motivo.

Problemas frecuentes

Síntoma	Causa y solución
No module named robomaster	Intérprete equivocado: elegir el de Python 3.8 (sección 1)
ConnectionRefusedError … debugpy	Depurador de VSCode colgado: ejecutar con Ctrl+F5 o «Run Python File»
FileNotFoundError serial_number.txt	Solo en networking: lanzar antes connect.py
No se encuentra el modelo	Copiar model_demo.pt en models/faster_rcnn_v2_aug_robusto_v1_lr001/ o indicar --modelo RUTA
El robot no conecta	Revisar la posición de la pestaña de conexión, la red del portátil y el cortafuegos (permitir Python en redes privadas); reiniciar el robot
La torreta apunta al lado contrario	SIGNO_YAW = -1 en circuito.py
Inferencia muy lenta	RESIZE en circuito.py, o --arquitectura faster_rcnn_v3 --modelo RUTA con ese checkpoint
No suenan los sonidos	Cambiar los números de SONIDOS (nota A de la sección 3)
Algo va mal con el robot en marcha	Ctrl+C: para el chasis, centra la torreta, apaga los LED y guarda el resumen